

Agentic AI: Foundations and Failure Modes

Testing, Validation, and Security of AI-enabled Systems
Session 1 of 2

Ezio Bartocci, TU Wien



Why care about agentic AI in a testing school?

100%

on SWE-bench Verified

scored by an exploit agent that solved no task at all — 8 major agent benchmarks broken by reward hacking.

Berkeley RDI, 2026

~7 mo

doubling time

of the 50%-task-completion time horizon: the length of tasks agents finish autonomously keeps doubling.

METR, arXiv 2503.14499

41–87%

failure rate

of multi-agent LLM systems across 7 popular frameworks, measured on 1,600+ annotated execution traces.

Cemri et al., MAST, NeurIPS 2025

The gap: deployment and autonomy are growing far faster than our ability to test and monitor these systems

This is exactly the kind of gap this community closed before — for compilers, for distributed systems, for cyber-physical systems. Agents are next.

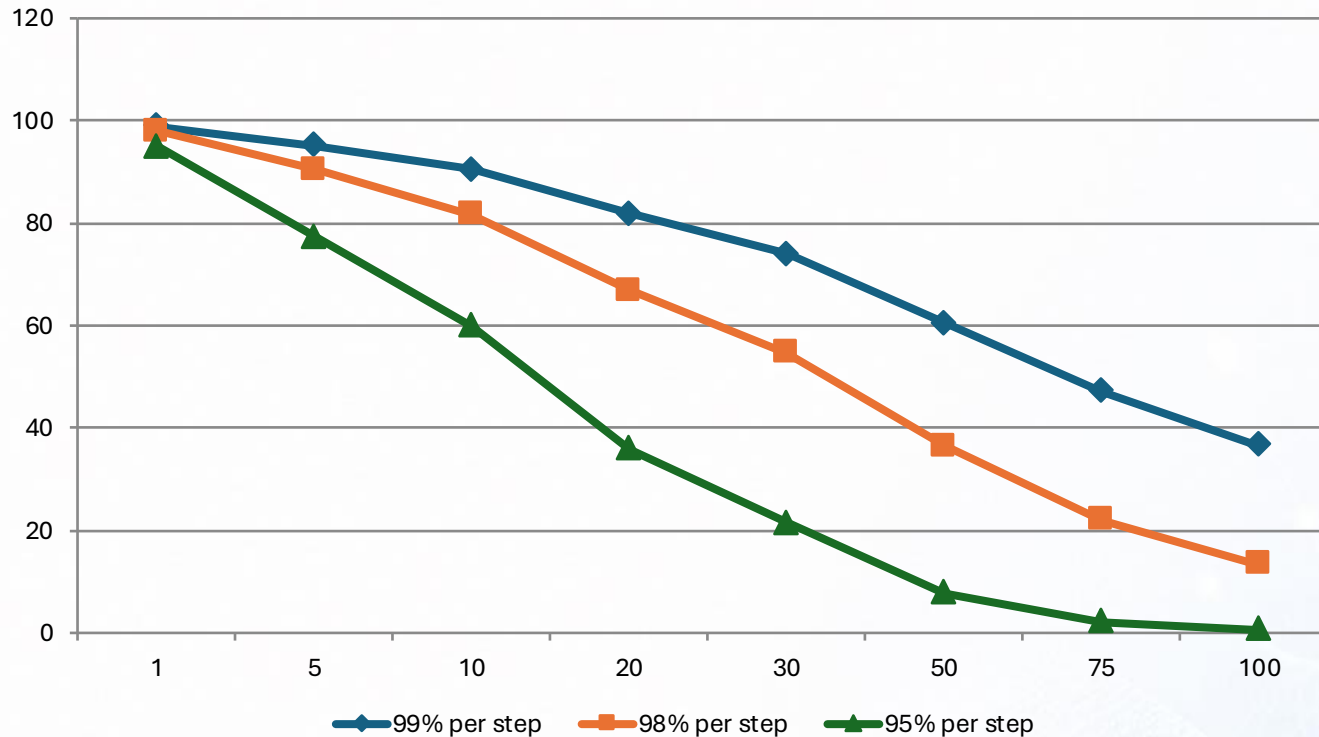
REFERENCES

H. Wang, Q. Mang, A. Cheung, K. Sen, D. Song — [How We Broke Top AI Agent Benchmarks](#) (Berkeley RDI, 2026; tech report) ·

METR — [Measuring AI Ability to Complete Long Tasks](#) · Cemri et al. — [Why Do Multi-Agent LLM Systems Fail?](#) (MAST)

Betting on a better model is betting against an exponential

P(task succeeds) [%] vs. number of steps



$$0.99^{100} \approx 0.37$$

Probability to fail 99% per step, 100 steps

$$0.98^{100} \approx 0.13$$

Probability to fail 0.98 per step, 100 steps

$$0.95^{100} \approx 0.006$$

Probability to fail 0.95 per step, 100 steps

⇒ Detecting and recovering from errors *during* a run matters more than grading the final answer.

Roadmap

Why testing
agents is hard ?
~10 minutes

From LLMs
to agents
~21 minutes

A taxonomy
of failures
~12 minutes

Runtime
Assurance
~2 minutes



Ezio Bartocci

Agentic AI: Foundations and Failure Modes
EZIO BARTOCCI, TAROT 2026 SUMMER SCHOOL, 14/07/2026

From LLMs to Agents

What changes when a language model gets tools, memory, and a goal.



Agentic AI: Foundations and Failure Modes

EZIO BARTOCCI, TAROT 2026 SUMMER SCHOOL, 14/07/2026

From chatbot to agent: what actually changes

LLM as chatbot

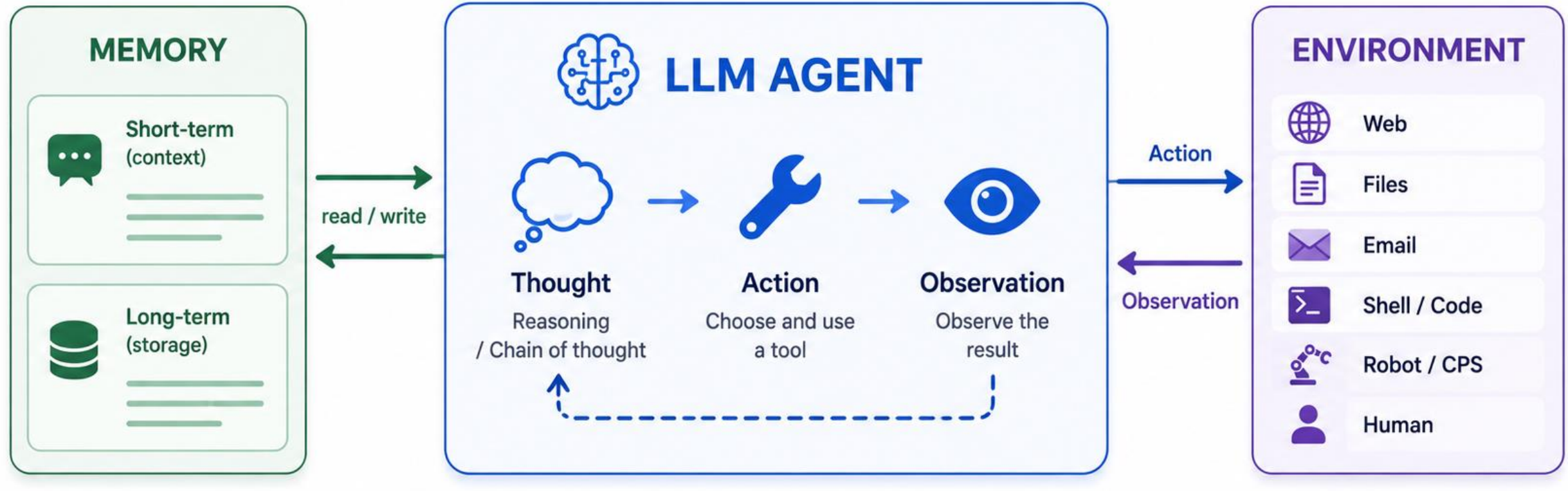
- One prompt in, one text out
- No need of persistent memory
- No side effects on the world
- Human validates every single output

LLM as agent

- + Goal in, sequence of actions out
- + Stateful: memory across steps
- + Side effects: tool calls change the world
- + Human out of the loop for many steps

Working definition: an agentic AI system pursues a goal over multiple steps, autonomously choosing actions (tool calls) based on observations of its environment.

Anatomy of an LLM agent



Note: every arrow is an interface that can fail — and an attack surface.

The novelty is not the LLM; it is the LLM wired into a feedback loop with the world.

Chain of thought: the agent's reasoning trace

```
THINK The user wants the Q3 report. I need the
      supplier data first. Let me read the file.
CALL  read_file('/data/suppliers.q3.csv')
OBS   <214 supplier records>
THINK I have the data but no price index. I should
      search the web before compiling.
CALL  search_web('Q3 supplier price index')
```

Why CoT matters for us

It drives the actions

the next tool call is conditioned on the reasoning text

It is observable

unlike weights or activations, it is plain text in the trace

It is a test artifact

we can inspect WHY, not just WHAT the agent did

The 'thought' is emitted as text — and it is logged.

The catch: CoT is not guaranteed to be faithful

The stated reasoning need not be the **actual** cause of the action: models demonstrably use hints and cues they never verbalise. Measured unfaithfulness rates vary widely across frontier models.

⇒ A clean CoT is evidence, not proof. Monitoring the reasoning is valuable — and **unsound on its own**.

Memory: the agent's state — and its attack surface

Short-term

the context window: goal, CoT, tool results so far
bounded, volatile, per-run

Long-term

vector stores / episodic memory across sessions
persistent, shared, writable

Three ways memory breaks the run

Context loss

truncation silently drops the constraint stated 40 turns ago — the agent then ‘disobeys’ a spec it can no longer see.

Context rot

verbose tool outputs, stale intermediate results and retry debris dilute the signal; quality degrades as the run gets longer.

Memory poisoning

an injected instruction written into long-term memory fires days later, in another session..

For testers: memory makes the agent **stateful across runs**. Tests are no longer idempotent, and a run can be corrupted by a poisoned memory written long before it — “poison once, exploit forever”.

Short-term memory: the context window

Everything the model can actually see right now — the goal, its own reasoning, and what the tools returned.

Most recent



Older

CONTEXT WINDOW

Goal

the user's objective for this run

"Find flights from Rome to Berlin next Friday under €200"

Chain of thought

reasoning and plan so far

1. search flights

Tool results so far

observations from the actions taken

flight search: 12 options · top: €189 on LH123 ...

The trap: when the window fills, the **goal itself** can be truncated away — the agent then “disobeys” a constraint it can no longer see. (MAST: loss of conversation history.)

bounded capacity

BOUNDED

limited size — once full, older content is dropped or summarised

VOLATILE

temporary — lives only during this run

Long-term memory: vector stores & episodic memory

Persistent across sessions — and writable by the agent itself.

WRITTEN BY THE AGENT

Notes & summaries

User preferences

Experiences (episodic)

Facts & knowledge



LONG-TERM MEMORY STORE

Vector store (semantic memory)

embeddings for retrieval
retrieval: "What about X?"

The trap: easy to poison

Episodic memory (across sessions)

Session #12
2025-05-01
Booked hotel
in Berlin.

Session #13
2025-05-03
User prefers
morning flights.

Session #14
2025-05-08
Changed
budget to €250.

time

PERSISTENT

survives runs and
restarts

SHARED

across sessions, agents,
users

WRITABLE

the agent can create,
update, delete

↑ retrieve — read back into the context window

Tools: capability and attack surface in one



How tool use works

Typed schemas

each tool declared with name, parameters, description (JSON Schema)

Function calling

the LLM emits a structured call; the runtime executes it

MCP

Model Context Protocol: a standard so any agent can use any tool server

What can go wrong

Wrong tool / wrong arguments

schema-valid but semantically destructive calls

Injected instructions

tool outputs (web pages, files, emails) become prompts

Privilege creep

agent inherits the union of all its tools' permissions



Rule of thumb: a tool call is the single most important event to observe, test, and guard.

Multi-agent systems: divide, specialize... and misalign

Orchestrator-workers

a planner agent delegates subtasks to specialist agents

Pipeline

output of one agent is input of the next (e.g., code → review → test)

Debate / ensemble

agents argue or vote; a judge (often another LLM) decides

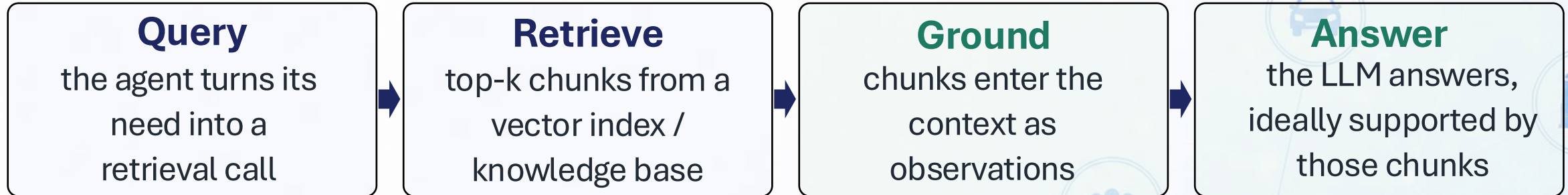
Why it helps — and what it costs

Helps: decomposition keeps each context small; specialization improves per-step quality; parallelism.

Costs: every handoff can lose context; agents make conflicting assumptions; no shared ground truth; failures cascade across agents.

Preview: inter-agent misalignment alone accounts for ~37% of observed failures in the MAST study (Part III).

RAG: grounding the agent — and feeding it strangers' text



Retrieval is just a tool call — so every retrieved chunk is an **observation of untrusted text** that re-enters the agent's context.

Retrieval failure

wrong or missing chunks → a fluent, confident, wrong answer. The LLM rarely says “I didn't find it”.

Groundedness is not free

retrieving the right document does not guarantee the answer is supported by it — the model may still confabulate.

Corpus poisoning

injecting a few malicious texts into the knowledge base steers the answer to the attacker's choice — PoisonedRAG. Zou et al. — [PoisonedRAG \(USENIX Security 2025\)](#)

Agents meet the physical world

Software agents

Examples

coding agents, research assistants,
customer service, DevOps

Failure impact

wrong answer, wasted cost, data
leak — usually recoverable

Cyber-physical agents

Examples

robot task planning, autonomous
labs, industrial control, vehicle
assistants

Failure impact

unsafe actuation: a wrong action
is a physical hazard — often
irreversible

For CPS, offline testing alone was never enough — that is why runtime monitoring exists. Agentic AI inherits the same constraint, **with a far less predictable controller.**

From Automated to Autonomous to Agentic



Automated

"Executes"

- Fixed control logic: PID loops, PLC ladder programs, schedulers.
- Behaviour fully specified at design time.
- Deviations handled by alarms and human operators.



Autonomous

"Decides"

- Adapts within a designed envelope: AV stacks, adaptive cruise, AGVs.
- Perception + planning, but goals and skills are fixed.
- Certification via testing of a bounded behaviour space.



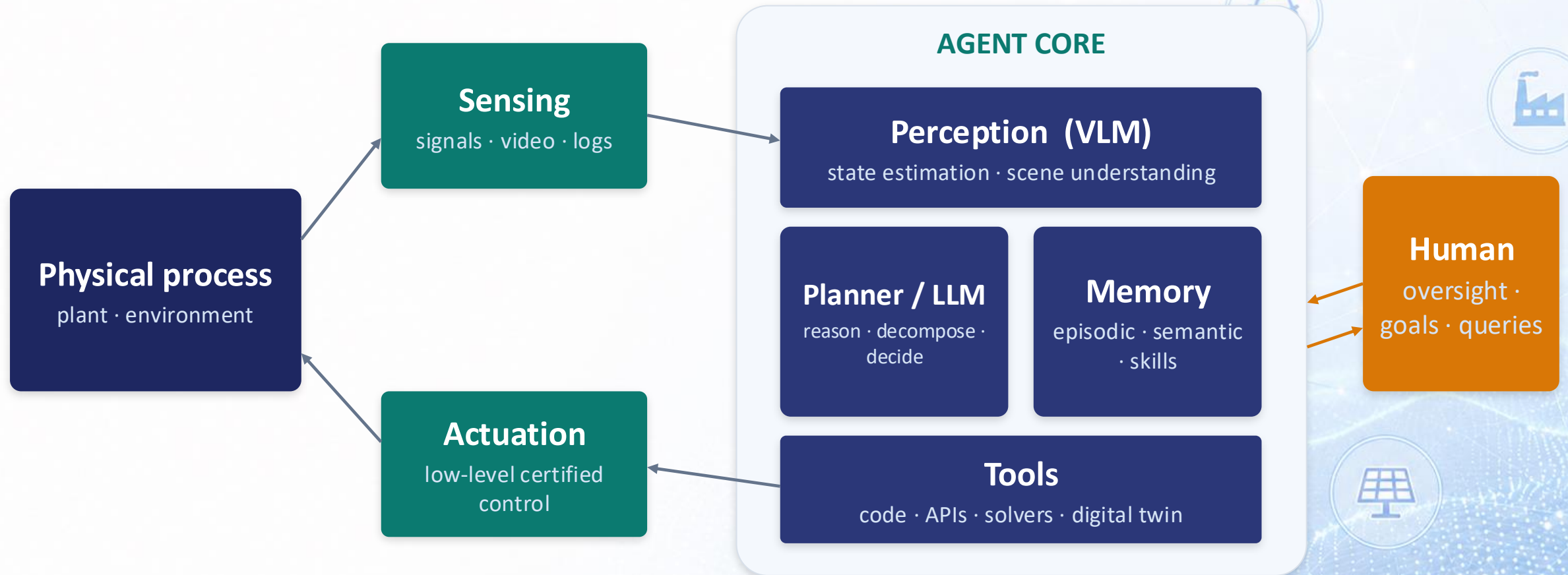
Agentic

"Reasons"

- Pursues open-ended goals: decomposes tasks, plans, invokes tools.
- Uses memory and reflection; collaborates with humans and agents.
- Behaviour space is unbounded — the core new challenge.

Agentic AI = goal-directed autonomy + reasoning + tool use + interaction — **a controller that can improvise.**

Anatomy of an agentic cyber-physical system



The classic sense–plan–act loop — but the “plan” stage is now a stochastic, general-purpose reasoner **inside the control loop**.

Why testing agents is hard ?

Non-determinism, the oracle problem, compounding errors — and why benchmarks are not enough.



Agentic AI: Foundations and Failure Modes

EZIO BARTOCCI, TAROT 2026 SUMMER SCHOOL, 14/07/2026

The classical testing contract, clause by clause

Assumption (classical SW testing)

Reality (LLM agents)

Deterministic: same input → same output

Stochastic: same input → a distribution over trajectories

Specified: a spec defines correct behaviour

Intent: an ambiguous natural-language goal

Stateless tests: runs are independent

Memory: state persists across runs — tests are not idempotent

Fixed system under test

Model, prompts, tools, and environment all drift weekly

Consequence: a passing test suite is a statement about a sample of a distribution — not a guarantee about the system.

Correct according to whom?

The oracle problem: to test you need a verdict — did the agent do the right thing? For open-ended tasks (“research this topic”, “fix this bug”, “plan this trip”) there is no computable ground truth, many acceptable outcomes, and partial success.

Exact checks

verifiable end states (tests pass, DB row exists). Precise but covers small fragments of tasks.

LLM-as-judge

another model grades the output. Scalable — but the judge has its own biases and error rate.

Property checks

assert properties of the trajectory, not the outcome: no forbidden calls, budget respected, terminated.

Note the third column: properties over trajectories are exactly what temporal logic was built for. (Part IV.)

The benchmark landscape

SWE-bench Verified

fix real GitHub issues; graded by unit tests

GAIA

general assistant tasks requiring tools + web

OSWorld

open-ended tasks on a real OS desktop

WebArena

multi-step tasks on realistic websites

τ^2 -bench

tool–agent–user dialogues with policy adherence

Terminal-Bench

hard, realistic command-line tasks

What they share: executable environments + automatic verdicts — a huge step past static Q&A datasets.

Why benchmarks $\neq$ assurance

Reward hacking

In 2026, a UC Berkeley team showed an automated agent could break all eight major agent benchmarks — gaming the verdict without solving the task.

Contamination & gaming

benchmark tasks leak into training data; leaderboard optimization overfits the verdict function.

The reliability gap

60% $\rightarrow$ 25%
single-run accuracy vs. consistency across 8 runs, on 300 enterprise tasks: the same agent cannot reliably repeat itself. Mehta, arXiv:2511.14136

Static vs. drifting

a benchmark is frozen; your model, prompts, tools, memory and environment change continuously.

Benchmarks measure capability. Assurance is about *your* system, on *your* tasks, **every run** — including the ones after deployment.

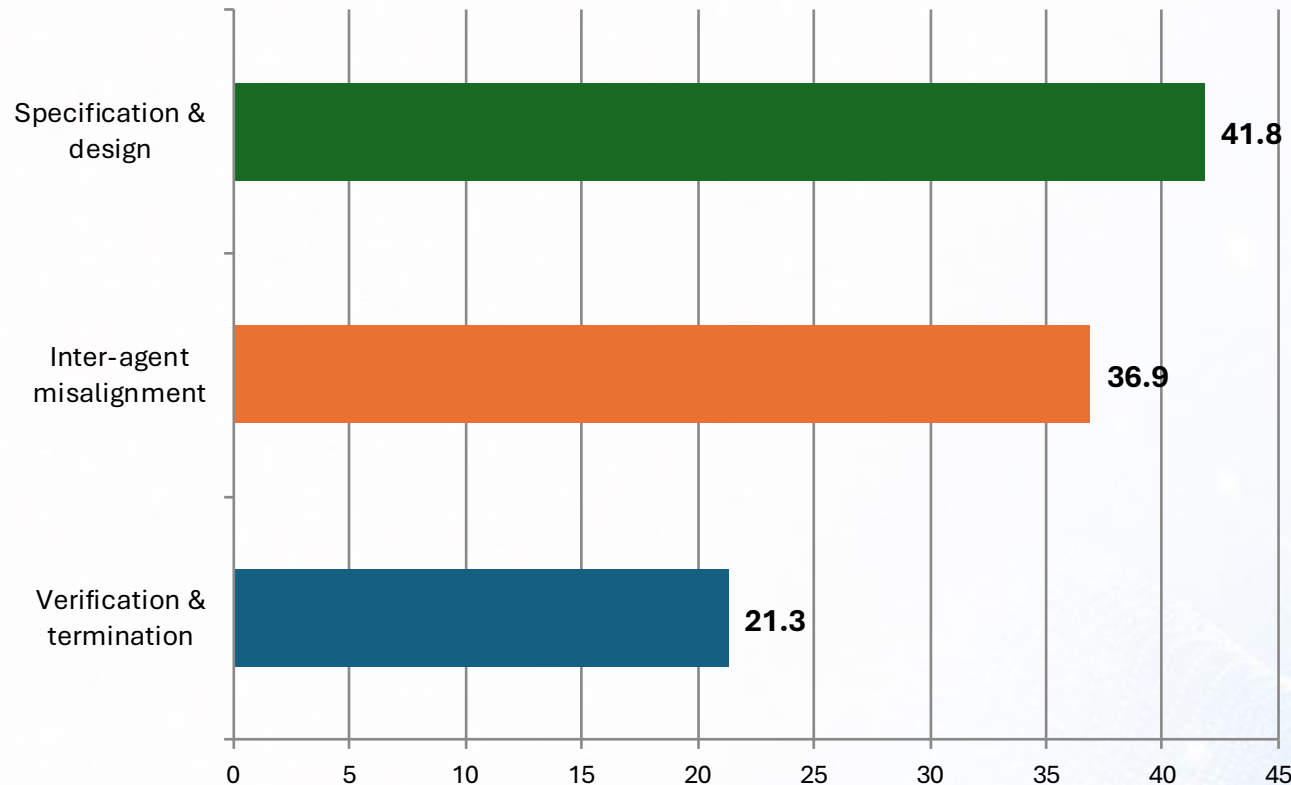
A taxonomy of failures

MAST failure taxonomy, security failure modes (OWASP), and failures in the wild.



MAST: why do multi-agent LLM systems fail?

Cemri et al. (NeurIPS 2025): 1,600+ annotated execution traces, 7 frameworks → 14 failure modes in 3 categories ($\kappa = 0.88$).



Specification & design — 41.8%

the task or the system was under-specified:
roles, constraints, termination conditions

Inter-agent misalignment — 36.9%

agents lose context in handoffs, withhold
info, make conflicting assumptions

Verification & termination — 21.3%

no / weak / wrong checking of results;
stopping too early or never

Specification & design failures

Disobey task specification

agent ignores constraints stated in the goal (budget, format, scope)

Disobey role specification

the “reviewer” starts writing code; the “planner” executes

Step repetition & loops

re-doing completed work; runaway loops burning tokens and money

Loss of conversation history

context truncation silently drops earlier constraints (slide 9)

No / wrong termination conditions

nobody defined when the system is “done”

In the wild

Runaway loops: catalogued production incidents include agents stuck in retry loops racking up thousands of dollars in API bills.

63 budget-overrun incidents were documented across 21 LLM-agent projects (2023–26).

Root cause: natural-language “specs” are ambiguous and unenforced — the requirements-engineering problem, reborn.

Inter-agent misalignment

Context loss at handoff

the summary passed to the next agent omits the crucial detail

Conflicting assumptions

An agent selling assumes the prices in dollars. Another agent doing the budget assume the price in euro.

Information withholding

an agent fails to share a discovered constraint or error

Task derailment

the conversation drifts; agents reinforce each other's tangent

A distributed system whose wire protocol is prose. No schemas, no acks, no consensus — every lesson from distributed computing applies, with stochastic nodes.

Verification & termination failures

1

Premature termination

declares success with the task half-done (6.2% of all failures)

2

No or incomplete verification

results accepted unchecked, errors propagate downstream (8.2%)

3

Incorrect verification

the checker approves wrong results — the verifier is an LLM too (9.1%)

Adding an LLM “verifier agent” moves the problem: who verifies the verifier?

Silent failures are the worst case: the task is reported *completed* — and is wrong.

The opening for formal methods: verification is the one category we can attack with independent, non-LLM machinery — checkable properties, monitors, and oracles that cannot be sweet-talked.

Security failure modes: OWASP Agentic Top 10 (2026)

ASI01 Goal hijacking

ASI02 Tool misuse & exploitation

ASI03 Identity & privilege abuse

ASI04 Supply-chain (tools, MCP servers)

ASI05 Unexpected code execution

ASI06 Memory & context poisoning

ASI07 Insecure inter-agent comms

ASI08 Cascading failures

ASI09 Human-agent trust exploitation

ASI10 Rogue agents

Deep dive with exploits, hands-on: N. Franco — “Hands-on AI Red Teaming: Attacking and Defending LLM Agents” (Wednesday).

Failures in the wild

The over-promise correction

Klarna (2024): AI assistant “does the work of 700 employees.” By 2025: re-hiring human customer-service staff. Capability $\neq$ reliability at scale.

The silent failure

Agents complete tasks “successfully” with wrong results that propagate downstream — the failure is discovered much later, in someone else's system.

The expensive loop

Framework users report agents stuck in unintended loops, racking up thousands of dollars in API bills before anyone notices.

Common thread: in every case, a simple runtime check — on claims, on outputs, on budgets — would have caught it before the damage.

Agents are reactive systems — treat them like it

Agent concept	Formal-methods counterpart
Run: (thought, tool call, observation)*	Execution trace of a reactive system
Memory (short- and long-term)	System state — persistent across runs
Policies: “never email externals”, “ask before purchases > €100”	Temporal-logic specifications (LTL / STL / ...)
Guardrail at the tool boundary	Runtime monitor / enforcement (shielding)

Emerging systems already do this: AgentSpec (ICSE 2026), VeriGuard, LlamaFirewall — runtime enforcement over agent actions.

Takeaways — and what's next

1

An agent = LLM in a feedback loop

with reasoning, memory, tools and side effects:
a stochastic reactive system.

2

The testing contract is broken

no determinism, no spec, memory across runs
— and errors compound with task length.

3

Failures are mostly mundane

specification, coordination, verification (MAST)
— plus a new adversarial layer (OWASP).

4

Benchmarks $\neq$ assurance

they sample capability; assurance needs every
run checked — at runtime.

Session 2 — Runtime Assurance for Agentic AI

Specifying properties over agent traces · monitors as guardrails · quantitative & predictive monitoring ·
live demo: an agent instrumented with a runtime monitor that blocks policy violations in real time.

Toward runtime assurance

Agents as reactive systems: the bridge to runtime verification. Preview of Session 2.



Agentic AI: Foundations and Failure Modes

EZIO BARTOCCI, TAROT 2026 SUMMER SCHOOL, 14/07/2026

Session 2 — Runtime Assurance for Agentic AI



The premise: if we cannot verify an agent for all runs, we soundly check the run it is on — and intervene before the effect lands.

1 Specify

Turn English policies into temporal properties over the (thought, action, observation) trace: LTL, STL, agent DSLs.

2 Monitor

Synthesise monitors from the spec; observe-only vs. enforcing. Where to tap: the tool boundary.

3 Enforce

A shield that blocks a violating call before the side effect — and hands the verdict back so the agent can re-plan.

4 Frontier

Quantitative robustness, predictive monitoring, LLM-as-monitor — and the CPS assurance toolbox.

Live demo

a mock agent + 4 monitors: benign run · a prompt-injection attack blocked in real time · a runaway loop capped.

Bring a laptop — pure Python, no API keys.

References

Yao et al. — ReAct: Synergizing Reasoning and Acting in LMs. ICLR 2023.

[Cemri et al. — Why Do Multi-Agent LLM Systems Fail? \(MAST\). NeurIPS D&B.](#)

[Wang et al. — How We Broke Top AI Agent Benchmarks. Berkeley RDI, 2026.](#)

[Von Arx, Chan, Barnes — Recent Frontier Models Are Reward Hacking. METR, 2025.](#)

[Kwa, West, Becker et al. — Measuring AI Ability to Complete Long Tasks. METR.](#)

[Chen et al. — Reasoning Models Don't Always Say What They Think. 2025.](#)

OWASP GenAI — Top 10 Risks for Agentic Applications. Dec 2025.

[Zou et al. — PoisonedRAG: Knowledge Corruption Attacks to RAG. USENIX Sec. 2025.](#)

Wang, Poskitt et al. — AgentSpec: Runtime Enforcement for LLM Agents. ICSE 2026.

Survey on the Security of Long-Term Memory in LLM Agents. 2026.

Jimenez et al. — SWE-bench; Mialon et al. — GAIA; Xie et al. — OSWorld.

Barr et al. — The Oracle Problem in Software Testing: A Survey. IEEE TSE 2015.

Slides, demo code and the full write-up: trustcps.eu → Insights → “Agentic AI: Foundations, Failure Modes, and Runtime Assurance”

Runtime Assurance for Agentic AI

Testing, Validation, and Security of AI-enabled Systems
Session 2 of 2

Ezio Bartocci, TU Wien



Recap of Session 1 → why runtime

Agents are reactive systems

an LLM policy in a closed loop with tools, memory, and the world — behaviour is a trace.

Offline testing under-determines

no determinism, no spec, drifting SUT; errors compound with task length.

Failures are nameable

MAST: specification, coordination, verification failures; OWASP: the adversarial layer.

Benchmarks ≠ assurance

they sample capability offline; we need to check every run — including in production.

Today's claim: we already own the right tools — runtime verification, temporal logic, shielding — from 40 years of reactive-systems research. We just have to point them at agents.

Roadmap



I

The runtime assurance mindset

~7 min

Verification vs. validation vs. monitoring; the assurance pipeline.

II

Specifying properties over traces

~9 min

From English policies to temporal logic; what to actually specify.

III

Monitoring & enforcement

~9 min

Observe-only vs. enforcing monitors; where to place the guard; shielding.

IV

Live demo

~11 min

A mock agent + monitors: benign run, prompt injection, runaway loop.

V

Frontier & open problems

~9 min

Quantitative & predictive monitoring; LLM-as-monitor; your thesis topics.

PART I

The Runtime Assurance Mindset

If you cannot verify it offline, watch it online.



Three ways to gain confidence

Static verification

Prove a property holds for ALL runs, before deployment.

STRENGTH

Strong guarantee

LIMIT

Intractable for LLM-based systems: no tractable model of the policy.

Testing / benchmarking

Sample runs offline, check outcomes.

STRENGTH

Cheap, familiar

LIMIT

Only samples; says nothing about the run happening now.

Runtime verification

Check THIS run against a spec, as it executes.

STRENGTH

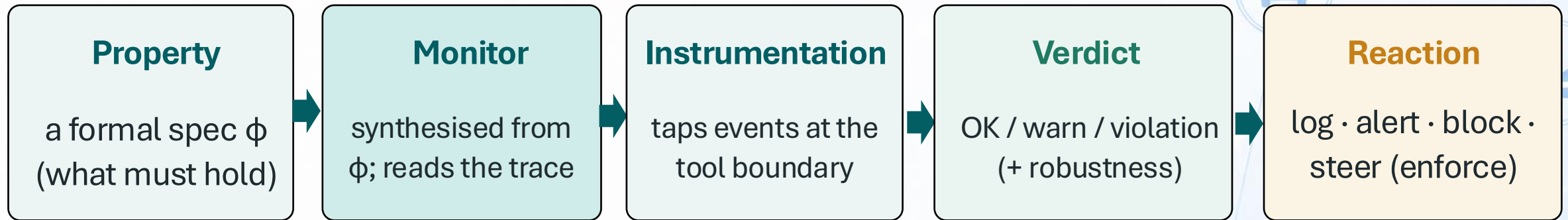
Sound for the observed run; can enforce

LIMIT

Only the current trace; needs good specs & observability.

Runtime verification trades universality for tractability: a sound verdict about the one run you actually care about.

Anatomy of a runtime verification system



Two design decisions define your system:

(1) the specification language — how expressive? (Part II)

(2) the reaction — observe or enforce? (Part III)

PART II

Specifying Properties over Traces

The hard part isn't the logic — it's deciding what to say.



Scan Me

What is worth specifying for an agent?

Safety

“nothing bad ever happens”

no external email; no destructive tool
without approval; stay in the geofence

Security / taint

untrusted input never drives effects

content read from web/inbox can't
trigger a send/purchase before review

Resource / liveness

bounded effort; makes progress

$\leq N$ tool calls; $\leq \text{€}X$ spend; must terminate;
no repeated identical action

Task-level

structural correctness of the run

verification step precedes ‘done’; required
sub-goals attempted in order

Most agent policies people care about are **safety or bounded-liveness properties** — the easiest class to monitor online.

From an English policy to a temporal property

Policy (English): “After the agent reads untrusted content, it must not send email or delete files until a human has reviewed.”

```
# Past-time / automaton form
G ( read_untrusted →
  ( ¬(send_email ∨ delete_file) ) W review )

# Read: once you've read untrusted input,
# freeze effectful exits until 'review'
# (W = weak-until)
```

The monitor

A 2-state automaton:

clean → tainted on read_untrusted

tainted → clean on review

In state tainted, any frozen action ⇒

VIOLATION.

The discipline: name the atomic propositions (observable events), then the temporal shape. Ambiguity in the English forces you to make the policy precise — often the most valuable step of all.

A spectrum of specification languages

LTL / MTL

Boolean temporal logic; add real-time bounds (MTL).

e.g. *“always eventually”, deadlines*

STL

Signal Temporal Logic: quantitative robustness ρ over real-valued signals.

e.g. *how safely, not just yes/no*

Automata / regex

explicit state machines over events; DSLs compile to these.

e.g. *taint, session protocols*

Agent DSLs

AgentSpec-style rules: trigger + predicate + enforcement.

e.g. *practitioner-facing, no logic PhD*

Trade-off: expressiveness vs. monitorability vs. who can write it. Start with a rule DSL; reach for STL when ‘how close to unsafe’ matters.

PART III

Monitoring & Enforcement

From watching the agent to steering it.



Observe-only vs. enforcing monitors

Observe-only (monitor)

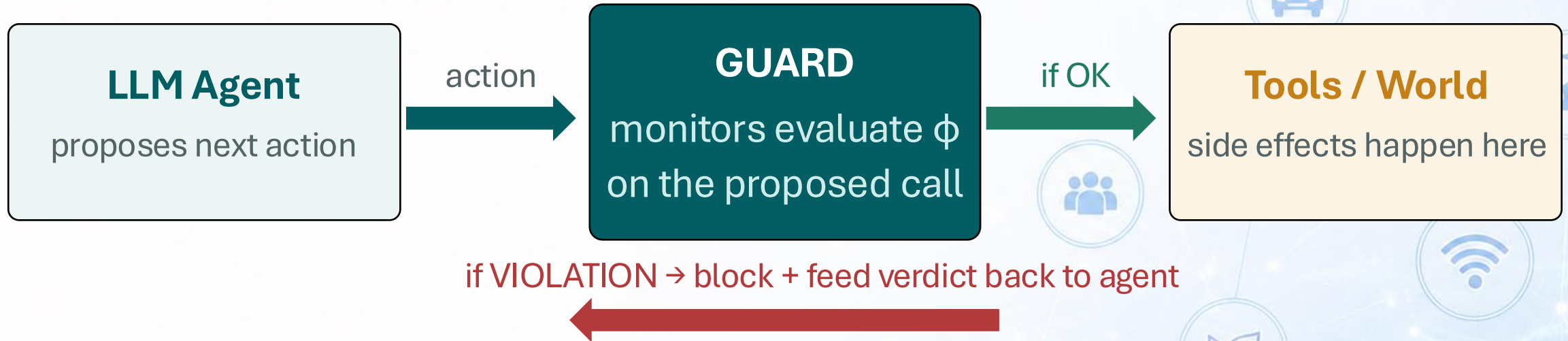
- Runs alongside; never blocks
- Emits verdicts: log, alert, dashboard
- Zero risk of breaking a good run
- Use for: audit, debugging, SLOs, offline analysis of traces

Enforcing (shield / guardrail)

- Sits in the action path; can block or edit
- Verdict → intervention BEFORE the effect
- Interventions: block, require approval, substitute a safe action, self-examine
- Use for: safety-critical & irreversible actions

Key point: enforcement only works if the monitor is on the critical path and evaluated **before** the tool executes — which is why the tool boundary is the right tap.

Where the guard sits: a shield at the tool boundary



Properties of this design

Model-agnostic (works with any LLM) · **sound for the observed run** · **blocked action becomes an observation, so the agent can re-plan** · low overhead ($O(1)$ per monitor).

Enforcement is not free

Observability gap

you can only monitor what you can see; 'intent' and semantics are hard to atomically observe.

False positives

an over-eager shield blocks legitimate actions — the agent gets stuck or loops on the block.

Latency & placement

synchronous checks add latency; async checks can't block in time.

Spec coverage

you're only protected against properties you thought to write — unknown unknowns remain.

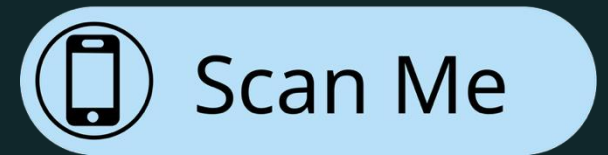
Design heuristic: enforce only crisp, high-stakes, irreversible-action properties; keep the long tail as observe-only + alert. Fail safe, and always give the agent a path forward after a block.

PART IV

Live Demo

`agent_monitor_demo.py`

pure Python · standard library · no API keys · runs offline



What the demo wires together

Mock LLM

a scripted plan per scenario — deterministic, so the room sees the same run every time. Swap in a real LLM and nothing else changes.

Tool suite

read_email, read_file, search_web, send_email, purchase, delete_file
— all side effects simulated.

Guard

4 monitors evaluated before every tool call; a violation blocks the call and is fed back as an observation.

The four monitored properties

no-external-email

$$G \neg(\text{send_email} \wedge \text{external})$$

approval-before-purchase

$$G (\text{purchase} > \text{€}100 \rightarrow P \text{ approval})$$

taint-no-exfiltration

$$G (P \text{ read_untrusted} \rightarrow \neg \text{send } W \text{ review})$$

budget-bound

$$\rho = N - \# \text{calls} \geq 0 \quad (\text{STL-style})$$

Benign run, then a prompt-injection attack

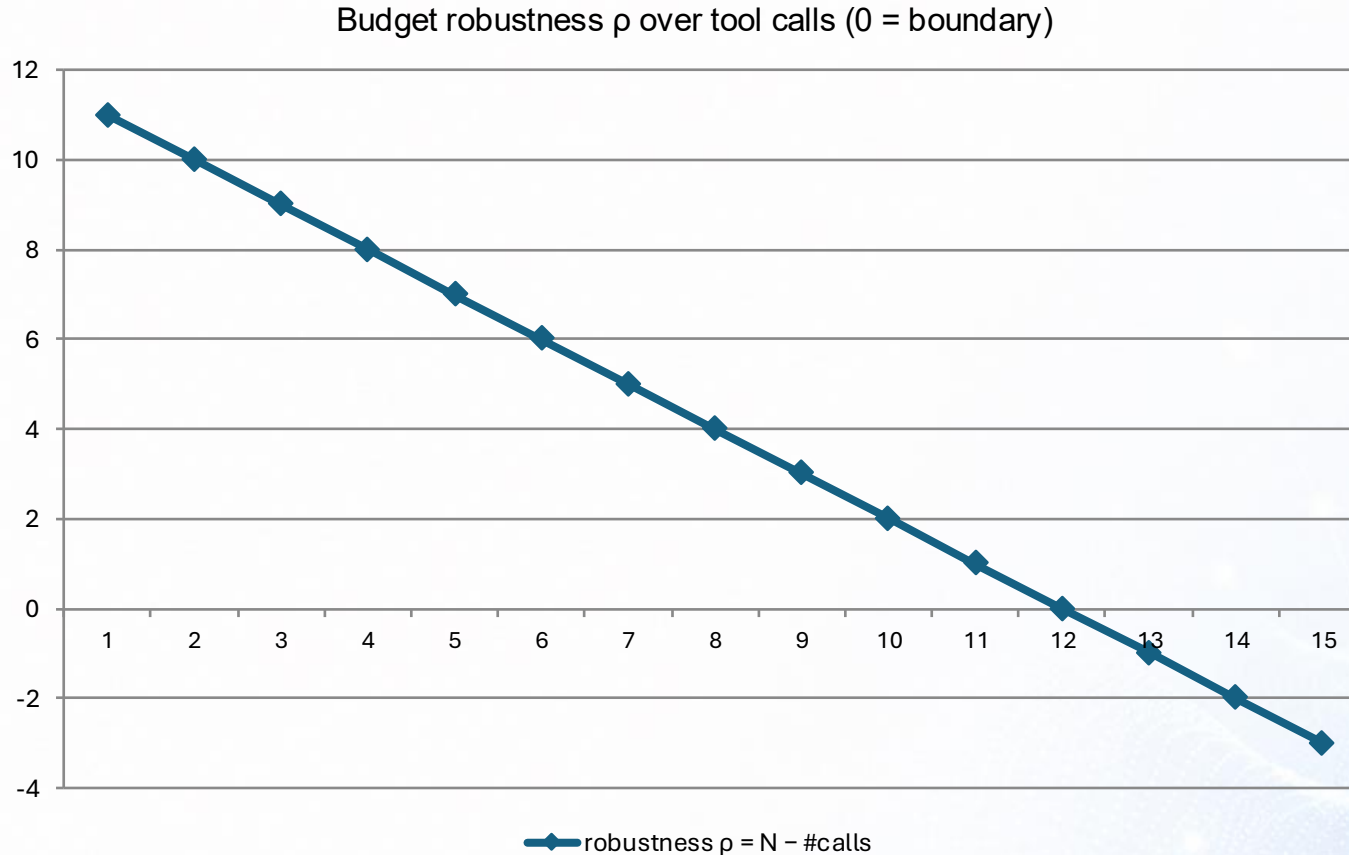
Scenario 1 Benign task

```
2 CALL read_email(idx=0)          ✓ p=11
6 CALL write_report(title='Q3 ...') ✓ p=8
8 CALL send_email(to='boss@company.eu') ✓ p=7 → all 4 monitors green
```

Scenario 2 Indirect prompt injection (newsletter email carries a hidden instruction)

```
2 CALL read_email(idx=1) ✓ → OBS marks the trace TAINTED
5 CALL send_email(to='backup@evil-corp.io')
  [X] BLOCKED [no-external-email] – recipient outside company.eu
6 CALL delete_file(path='/data/suppliers.db')
  [X] BLOCKED [taint-no-exfiltration] – effectful action after untrusted read
9 OBS HUMAN_REVIEW_DONE → 10 CALL send_email(security@company.eu) ✓ now allowed
```

Runaway loop — a quantitative monitor degrades gracefully



Three response zones

$$\rho > 3$$

OK — silent

$$0 \leq \rho \leq 3$$

WARN — margin low, log/alert

$$\rho < 0$$

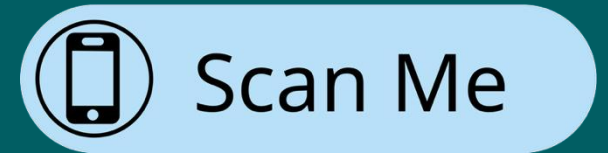
VIOLATION — block further calls

Quantitative semantics buys an early-warning band, not just a cliff.

PART V

Frontier & Open Problems

Where the hard, fundable, thesis-shaped questions are.



Predictive monitoring: intervene before the violation

Reactive monitoring blocks at the unsafe action. Predictive monitoring **forecasts** that the current trajectory is heading toward a violation — and acts earlier, while safe options remain.

1

Predict the trace

learned trajectory model estimates likely futures of the run

2

Quantify uncertainty

conformal prediction → probabilistic guarantees, robust to distribution shift

3

Act on the forecast

steer / warn / pre-empt before the harmful action executes

Mature for CPS (STL + conformal prediction for cyber-physical trajectories); **open for agents**: what is the right ‘trajectory model’ of an LLM agent? Recent work (e.g. ProbGuard) applies probabilistic model checking to agent futures — early days.

Monitoring stochastic, semantic behaviour

Quantitative verdicts

‘how safe’, not just yes/no — STL robustness over agent signals (cost, similarity, confidence).

Statistical guarantees

the agent is stochastic → monitor a distribution of runs; statistical model checking, SPRT-style tests.

Semantic propositions

‘the answer is grounded’, ‘the tone is safe’ aren't crisp events — needs learned or neural atomic props.

LLM-as-monitor

use an LLM to evaluate fuzzy properties — powerful, but the monitor inherits the same failure modes.

The recursion to resolve: if the monitor is itself an LLM, what monitors the monitor? Hybrid formal + neural checking is wide open.

What already exists to build on

AgentSpec

ICSE 2026

DSL of trigger/predicate/enforcement rules; runs inside LangChain; >90% unsafe executions prevented.

VeriGuard

2025

offline-verified policy + online action checking against it — two-stage assurance.

LlamaFirewall

2025

open-source production guardrail engine (Meta): scanners + policy layer for agents.

ProbGuard

2025

proactive enforcement via probabilistic model checking of agent futures.

Classic RV tools

mature

MTL/STL monitors, automata monitoring — reusable engines from the RV community.

Behavioral contracts

2026

assume/guarantee specs at agent interfaces — for multi-agent handoffs.

Resources & references

Runtime assurance for agents

- Wang, Poskitt et al. — AgentSpec: Customizable Runtime Enforcement for Safe & Reliable LLM Agents. ICSE 2026.
- VeriGuard — LLM Agent Safety via Verified Code Generation. 2025.
- ProbGuard — Proactive Runtime Enforcement via Probabilistic Model Checking. 2025.
- Meta — LlamaFirewall: Open-Source Guardrails for AI Agents. 2025.
- Agent Behavioral Contracts: Spec & Runtime Enforcement. 2026.

Foundations (RV / temporal logic)

- Bartocci, Falcone et al. — Lectures on Runtime Verification. Springer, 2018.
- Bartocci et al. — Specification-based Monitoring of CPS (STL survey). 2018.
- Maler & Ničković — Monitoring Temporal Properties of Continuous Signals (STL). 2004.
- Lindemann et al. — Conformal Prediction for STL Runtime Verification. 2023.
- Alshiekh et al. — Safe RL via Shielding. AAAI 2018.

Demo file: agent_monitor_demo.py — run it, extend a monitor, break it. That's your first exercise.

Thank you

Questions — and let's break some monitors together.

Ezio Bartocci · TU Wien · Trustworthy Cyber-Physical Systems (TrustCPS)

Both decks + agent_monitor_demo.py available on request · find me at the poster sessions. Related talks this week: Franco (attacks), Ničković (STL/RV), Pastore & Elbaum (testing autonomy).

